



Faculdade de Informática e Administração Paulista

Análise e Desenvolvimento de Sistemas

DOMAIN DRIVEN DESIGN USING JAVA

Sprint 4

Bridge Care

Turma do Bem

Pedro B. RM567478 | Lucas D. RM567600 | Alan C. RM567033

24 de Maio de 2026

Sumário

Objetivo do projeto	3
Breve Descrição das Principais Funcionalidades da Solução	4
Explicação dos Métodos Lógicos Desenvolvidos (Regras de Negócio):	5
DOCUMENTAÇÃO DE ENDPOINTS	8
Protótipo do Sistema	11
Modelo de Entidade-Relacionamento	14
Modelo-Relacional	15
Diagrama de classes	16
Procedimentos para rodar a aplicação (API Backend)	17

Objetivo do projeto

Objetivo:

O objetivo deste projeto é desenvolver uma API RESTful robusta para nosso projeto, denominado Bridgecare, projetada para otimizar, digitalizar e gerenciar o fluxo de atendimento da ONG Turma do Bem. A solução visa facilitar a conexão entre jovens em situação de vulnerabilidade social e dentistas voluntários, agilizando os processos de triagem, cadastro de beneficiários, programas sociais e o gerenciamento de consultas odontológicas.

Escopo do Projeto:

O escopo técnico abrange a construção do backend em Java, utilizando o framework Quarkus e aplicando os princípios de *Domain Driven Design* (DDD) para a modelagem do domínio. A solução contempla a persistência de dados em um banco de dados relacional (Oracle) e o gerenciamento de 9 entidades principais (Beneficiário, Pré-Beneficiário, Dentista, Consulta, Triagem, Solicitante, Programa Social, Endereço e o Relacionamento Triagem/Pré-Beneficiário). O sistema disponibiliza operações CRUD completas através de *endpoints* HTTP estruturados, garantindo regras de negócio consistentes (como a promoção automática de pré-beneficiários a beneficiários) e uma arquitetura escalável para suprir as necessidades operacionais da ONG.

Breve Descrição das Principais Funcionalidades da Solução

Principais Funcionalidades Implementadas:

- **Gestão de Beneficiários e Solicitantes:** Operações completas de CRUD (Criação, Leitura, Atualização e Eliminação) para gerir o registo de jovens que necessitam de atendimento odontológico e dos seus respectivos responsáveis.
- **Controle de Triagem:** Sistema de registo de avaliações, cruzando os dados clínicos preliminares e associando-os a um endereço específico e aos pré-beneficiários.
- **Agendamento e Histórico de Consultas:** Funcionalidade para registar e consultar o histórico médico, as datas, os horários e a alocação de dentistas voluntários para cada beneficiário.
- **Gestão de Dentistas e Programas Sociais:** Módulos dedicados à administração dos profissionais de saúde voluntários e dos programas de auxílio vinculados à Turma do Bem.

Explicação dos Métodos Lógicos Desenvolvidos (Regras de Negócio):

Para garantir a integridade dos dados e automatizar os processos da ONG, foram implementados métodos com regras de negócio rigorosas:

1. Validação e Promoção de Status (**PreBeneficiarioB0.atualizarStatus**)

- **Lógica:** Este método verifica se o novo estado fornecido é válido e não está vazio. A regra de negócio principal estipula que, se o pré-beneficiário for avaliado positivamente e o seu estado for alterado para Aprovado ("AP"), o sistema cria automaticamente um registo definitivo na tabela de **Beneficiario**. Isto promove o jovem para a fase de tratamento sem necessidade de intervenção manual dupla.

```
public void atualizarStatus(int id, String novoStatus) throws RegraNegocioException, SQLException {  
    if (novoStatus == null || novoStatus.trim().isEmpty()) {  
        throw new RegraNegocioException("O novo status não pode ser vazio.");  
    }  
  
    preBeneficiarioDao.atualizarStatus(id, novoStatus);  
  
    if ("AP".equals(novoStatus)) {  
        Beneficiario novoBeneficiario = new Beneficiario();  
        novoBeneficiario.setIdPreBeneficiario(id);  
        beneficiarioDao.cadastrar(novoBeneficiario);  
    }  
}
```

2. Orquestração Transacional de Registo (**PreBeneficiarioDAO.cadastrar**)

- **Lógica:** Garante a consistência estrutural através de uma transação manual na base de dados (`conn.setAutoCommit(false)`). O método insere primeiramente o endereço, recupera o ID gerado pelo Oracle e, apenas se houver sucesso, registra o pré-beneficiário e cria a relação de triagem. Se alguma etapa falhar, ocorre um `rollback` total, impedindo a criação de dados órfãos ou incompletos.

```
public void cadastrar(PreBeneficiario pb) throws SQLException { // Pedro Begam
    String sqlPre = "INSERT INTO T_BC_PRE_BENEFICIARIO (nm_pre_beneficiario, dt_nascimento, cpf_pre_beneficiario, sx_pre_beneficiario, ds_problema_dentario, st_situacao) VALUES (?, ?, ?, ?, ?, ?)";
    String sqlUpdateSolicitante = "UPDATE T_BC_SOLICITANTE SET st_solicitante = 'I' WHERE id_solicitante = ?";
    String sqlRelacaoTriagem = "INSERT INTO T_BC_REL_TRIAGEM_PRE_BENEF (id_triagem, id_pre_beneficiario) VALUES (?, ?)";

    Connection conn = null;
    try {
        conn = DatabaseConfig.getConnection();
        conn.setAutoCommit(false); // Transação iniciada

        EnderecoDAO enderecoDAO = new EnderecoDAO();
        int idEnderecoGerado = enderecoDAO.cadastrar(pb.getEndereco(), conn);
        int idPreBeneficiarioGerado = 0;

        try (PreparedStatement ps = conn.prepareStatement(sqlPre, new String[]{"ID_PRE_BENEFICIARIO"})) {
            ps.setString(1, pb.getNmPreBeneficiario());
            ps.setDate(2, java.sql.Date.valueOf(pb.getDtNascimento()));
            ps.setString(3, pb.getCpfPreBeneficiario());
            ps.setString(4, pb.getSxPreBeneficiario());
            ps.setString(5, pb.getDsProblemaDentario());
            ps.setString(6, pb.getStSituacao());
            ps.setInt(7, pb.getIdProgramaSocial());
            ps.setInt(8, idEnderecoGerado);
            ps.setInt(9, pb.getIdSolicitante());
            ps.executeUpdate();

            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) {
                    idPreBeneficiarioGerado = rs.getInt(1);
                }
            }
        }

        if (idPreBeneficiarioGerado == 0) {
            throw new SQLException("Falha ao cadastrar Pré-Beneficiário, ID não obtido.");
        }

        try (PreparedStatement psRel = conn.prepareStatement(sqlRelacaoTriagem)) {
            psRel.setInt(1, pb.getIdTriagem()); // Captura o valor idTriagem enviado pelo React
            psRel.setInt(2, idPreBeneficiarioGerado); // Usa o ID gerado no passo anterior
            psRel.executeUpdate();
        }

        try (PreparedStatement psUpd = conn.prepareStatement(sqlUpdateSolicitante)) {
            psUpd.setInt(1, pb.getIdSolicitante());
            psUpd.executeUpdate();
        }

        conn.commit();

    } catch (SQLException e) {
        if (conn != null) conn.rollback(); // Cancela tudo se houver falhas
        throw e;
    } finally {
        if (conn != null) {
            conn.setAutoCommit(true);
        }
    }
}
```

3. Validação de Prontuário Médico (**ConsultaB0.preencherProntuario**)

- **Lógica:** Impõe restrições de qualidade aos dados médicos inseridos. O método impede que sejam guardados prontuários vazios, nulos ou excessivamente curtos (exigindo um mínimo de 10 caracteres). Isto garante que o dentista forneça sempre um histórico médico detalhado e útil para as próximas fases do tratamento.

```
public void preencherProntuario(int idConsulta, String texto) throws RegraNegocioException, SQLException { 1 usage 2
    if (texto == null || texto.trim().isEmpty()) {
        throw new RegraNegocioException("O texto do prontuário não pode estar vazio.");
    }
    if (texto.length() < 10) {
        throw new RegraNegocioException("O prontuário deve conter detalhes suficientes (mínimo de 10 caracteres).");
    }
    dao.atualizarProntuario(idConsulta, texto);
}
```

4. Controlo de Estado de Voluntários (**DentistaB0.mudarStatus**)

- **Lógica:** Antes de efetuar qualquer alteração de estado na base de dados, o método filtra e bloqueia entradas incorretas. Apenas permite a transição do estado do dentista voluntário para Ativo ("A") ou Inativo ("I"), lançando uma exceção personalizada (**RegraNegocioException**) caso a API receba parâmetros inválidos.

```
public void mudarStatus(int id, String status) throws RegraNegocioException, SQLException { 1 usage 2 Pedro Begalli
    if (status == null || (!status.equalsIgnoreCase("A") && !status.equalsIgnoreCase("I"))) {
        throw new RegraNegocioException("Status inválido. Use 'A' para Ativo ou 'I' para Inativo.");
    }

    Dentista dentista = dao.buscarPorId(id);
    if (dentista == null) {
        throw new RegraNegocioException("Dentista não encontrado.");
    }

    dao.atualizarStatus(id, status.toUpperCase());
}
```

DOCUMENTAÇÃO DE ENDPOINTS

Documentação Oficial de Endpoints — API BridgeCare

Especificação Revisada de Arquitetura, Rotas, Verbos HTTP e Códigos de Status

Este documento contém o mapeamento integral dos recursos expostos pelo ecossistema do backend BridgeCare baseado na sua especificação OpenAPI 3.0 atualizada.

1. Pré-Beneficiários e Triagens

Gestão de fila de espera, mutirões, avaliações iniciais e reagendamentos.

Método	Rota / Endpoint	Descrição Operacional	Status HTTP
GET	/pre-beneficiarios	Listar todos os pré-beneficiários na fila de espera	200 OK
POST	/pre-beneficiarios	Cadastrar nova solicitação de pré-beneficiário	201 Created
PUT	/pre-beneficiarios/{id}	Atualizar dados completos da ficha médica/pessoal	200 OK
DELETE	/pre-beneficiarios/{id}	Excluir ou recusar registro de pré-beneficiário	204 No Content
PATCH	/pre-beneficiarios/{id}/status	Atualizar status situacional (ex: Aprovar cadastro)	200 OK
GET	/triagens/proximas	Listar as próximas triagens agendadas	200 OK
POST	/triagens	Agendar mutirão ou evento de triagem	201 Created
PATCH	/rel-triagem/pre-beneficiario/{id}/reagendar	Reagendar um pré-beneficiário para nova triagem	200 OK

2. Consultas e Beneficiários

Módulo central para marcação de consultas, controle clínico e pacientes aprovados.

Método	Rota / Endpoint	Descrição Operacional	Status HTTP
GET	/beneficiarios	Listar todos os beneficiários ativos	200 OK 500 Error

BridgeCare API — Especificação Técnica Atualizada

1

Método	Rota / Endpoint	Descrição Operacional	Status HTTP
PUT	/beneficiarios/{id}	Atualizar um beneficiário existente	200 OK 500 Error
DELETE	/beneficiarios/{id}	Remover um beneficiário por ID	204 No Content 500 Error
GET	/consultas	Listar todas as consultas gerais	200 OK 500 Error
POST	/consultas	Agendar uma nova consulta	201 Created 500 Error
GET	/consultas/dentista/{id}/proximas	Listar as próximas consultas de um dentista específico	200 OK
GET	/consultas/dentista/{id}/pendentes	Listar consultas pendentes (sem prontuário) de um dentista	200 OK
GET	/consultas/beneficiario/{id}/proximas	Listar as próximas consultas de um beneficiário	200 OK
GET	/consultas/beneficiario/{id}/historico	Listar o histórico de consultas de um beneficiário	200 OK
PATCH	/consultas/{id}/prontuario	Preencher o prontuário de uma consulta realizada	200 OK 400 Bad Req
PATCH	/consultas/{id}/recomendacao	Salvar recomendação para a consulta	200 OK 400 Bad Req

3. Dentistas, Solicitantes e Endereços

Gerenciamento dos profissionais parceiros, entidades solicitantes e cadastros espaciais.

Método	Rota / Endpoint	Descrição Operacional	Status HTTP
GET	/dentistas	Listar todos os dentistas parceiros	200 OK
GET	/dentistas/{id}	Buscar informações de um dentista específico pelo ID	200 OK 404 Not Found
PATCH	/dentistas/{id}/status	Alterar status de ativação do dentista (Ativo/Inativo)	200 OK 400 Bad Req
GET	/solicitantes	Listar todos os solicitantes ativos (ONGs, Responsáveis)	200 OK
POST	/solicitantes	Cadastrar um novo solicitante inicial	201 Created
PUT	/solicitantes/{id}	Atualizar contatos do solicitante	200 OK
DELETE	/solicitantes/{id}	Remover solicitante do sistema	204 No Content
GET	/enderecos	Listar endereços cadastrados no sistema	200 OK
POST	/enderecos	Cadastrar um novo endereço (Triagem ou Residencial)	201 Created
PUT	/enderecos/{id}	Atualizar dados de um endereço existente	200 OK

Protótipo do Sistema

Solicitar Atendimento

Preencha os dados abaixo e nossa equipe entrará em contato em breve.

QUAL SEU NOME COMPLETO?

Ex: João Silva

NOME DO SEU RESPONSÁVEL (SE HOUVER)

Nome do pai, mãe ou tutor

NECESSITA DE ATENDIMENTO EM LIBRAS?

☐ Sim ☒ Não

TELEFONE

(11) 99999-9999

MELHOR E-MAIL

seu@email.com

Enviar Solicitação

O Fluxo de Entrada (Cadastro de Solicitante/Pré-Beneficiário)

- **1. Ação no Frontend:** O usuário preenche o formulário na interface web. Ao submeter, o React (Vite) empacota esses dados em um objeto JSON e realiza uma requisição HTTP do tipo **POST** para a nossa API."
- **2. Processamento na API (Backend):**
 - O **SolicitanteResource** intercepta a requisição, envia o DTO para o **SolicitanteBO** aplicar as regras de negócio e validações. Estando tudo correto, o **SolicitanteDAO** executa o comando de inserção.



A Visão da Operação (Dashboard do Atendente / Triagem)

- **1. Ação no Frontend:** Ao acessar o painel de controle, o sistema do atendente precisa carregar a fila de jovens aguardando aprovação. O frontend dispara uma requisição **GET** ao ser montado na tela.
- **2. Processamento na API (Backend):**
 - A API recebe a requisição de leitura. O **TriagemDAO** realiza uma consulta relacional no banco para trazer apenas os cadastros com status pendente, passando pelo **TriagemBO** e devolvendo o JSON estruturado para o front.

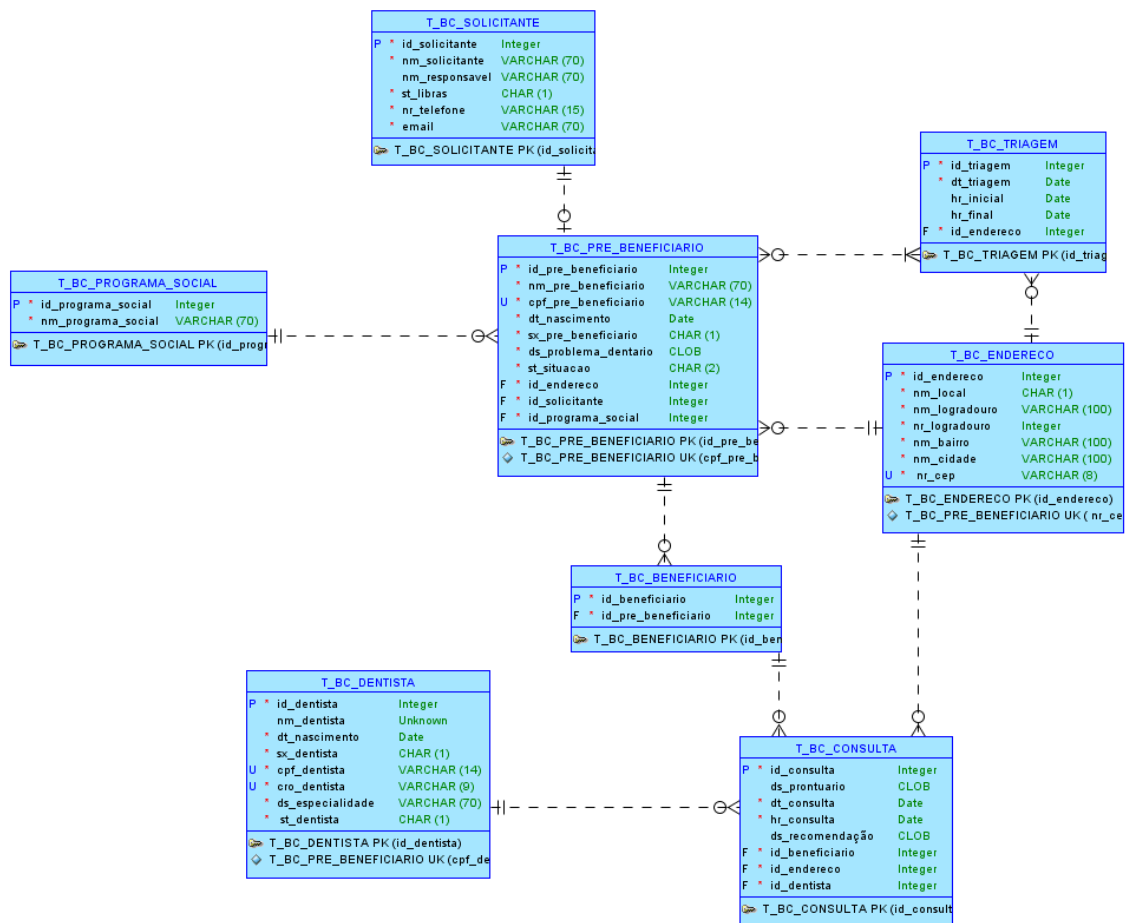
The screenshot displays the BridgeCare web application interface. At the top, there is a navigation bar with links: 'Quem somos', 'Sobre o projeto', 'Mini jogos', 'FAQ', and 'Contato'. On the right side of the navigation bar, there are two buttons: 'Quero doar!' (I want to donate) and 'Conecte-se' (Log in). On the left side, there is a sidebar menu with options: 'Visão Geral' (Overview), 'Minha Agenda' (My Agenda), 'Agendar Consulta' (Schedule Consultation - highlighted in blue), 'Prontuários Pendentes' (Pending Records), and 'Solicitar Materiais' (Request Materials). The main content area is titled 'Agendar Nova Consulta' (Schedule New Consultation) and includes the instruction 'Vincule um paciente da ONG à sua agenda marcando data e endereço.' (Link an NGO patient to your agenda by marking date and address). The form contains the following fields: 'BENEFICIÁRIO (PACIENTE)' (Beneficiary (Patient)) with a dropdown menu 'Selecione um paciente...' (Select a patient...); 'DATA DA CONSULTA' (Consultation Date) with a text input 'dd/mm/aaaa' and a calendar icon; 'HORA' (Time) with a text input '--:--' and a clock icon; and 'CONSULTÓRIO DE ATENDIMENTO' (Consultation Room) with a dropdown menu 'Selecione o local da consulta...' (Select the consultation location...). At the bottom of the form is a large blue button labeled 'CONFIRMAR AGENDAMENTO' (Confirm Scheduling). In the bottom left corner, there is a user profile for 'Daniela Freitas' with 'CRO: 45678-SP'.

O Fluxo de Atendimento (Dashboard do Dentista e Consulta).

- **1. Ação no Frontend:** O dentista voluntário registra uma nova avaliação para um beneficiário aprovado. É feito um **POST** (ou **PUT** para atualizações) através do painel.
- **2. Processamento na API (Backend):**
 - A chamada atinge o **ConsultaResource**. O **ConsultaB0** garante que o beneficiário associado é válido e que o dentista está ativo na plataforma, e então o **ConsultaDAO** salva o registro do atendimento.

Modelo de Entidade-Relacionamento

Modelo-Lógico



Modelo-Relacional

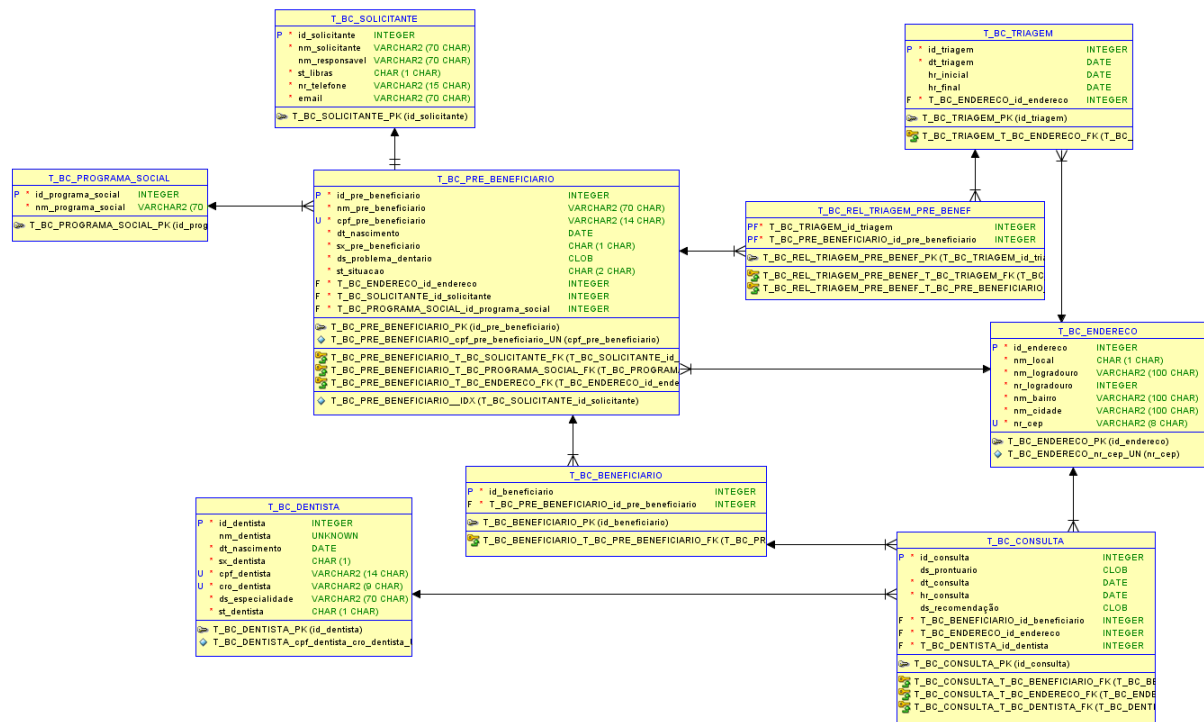
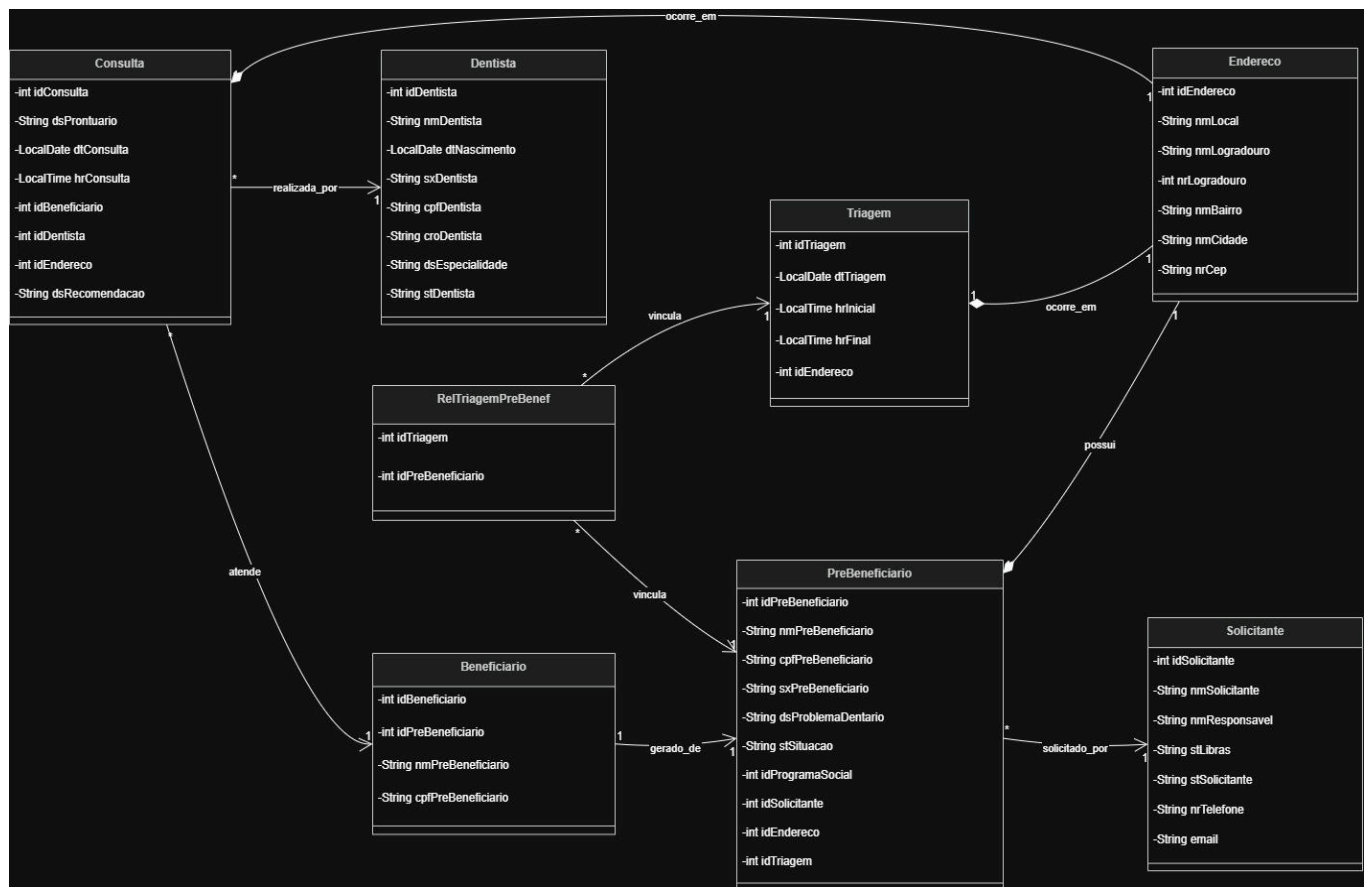


Diagrama de classes



Procedimentos para rodar a aplicação (API Backend)

Para a execução correta da API do projeto BridgeCare, siga as instruções abaixo:

1. Ferramentas e Versões Necessárias

- **Java JDK:** Versão 21.
- **IDE Recomendada:** IntelliJ IDEA, Eclipse IDE ou VS Code.
- **Framework:** Quarkus (não é necessário instalar globalmente, pois o projeto utiliza o Maven Wrapper).
- **Banco de Dados:** Oracle Database.

2. Configuração das Credenciais do Banco de Dados Por questões de segurança e boas práticas corporativas, as credenciais de acesso ao banco de dados não são versionadas no código-fonte. O sistema está preparado para receber essas configurações

Arquivo de Propriedades (Recomendado para desenvolvimento local) Crie um arquivo chamado `database.properties` dentro do diretório `src/main/resources/` e adicione as seguintes linhas com as credenciais do banco Oracle:

```
db.url=jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL
db.user=RM567478
db.pass=270401
```

3. Execução do Projeto O projeto utiliza o Maven Wrapper, o que facilita a execução sem necessidade de instalar o Maven manualmente. Abra o terminal na raiz do projeto (onde está o arquivo `pom.xml`) e execute o comando abaixo para iniciar a API em modo de desenvolvimento (Live Coding):

- **No Windows:**

```
mvnw.cmd compile quarkus:dev
```

- **No Linux/Mac:**

```
./mvnw compile quarkus:dev
```

Após a inicialização, a API estará rodando e pronta para receber requisições, normalmente acessível na porta padrão `8080`.